

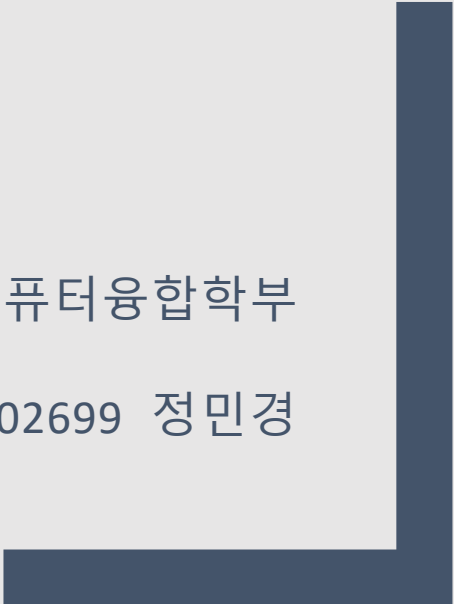


영상처리

14 주차 보고서

컴퓨터융합학부

202102699 정민경



1. 과제 설명

함수이름	my_median_filtering ()
코드설명	Filtering 방법 중 mask size 크기 안에서 src 의 값을 모두 정렬해 그 중 중앙값을 그 자리의 값으로 교체하는 방법을 구현한 코드이다. Median filtering 은 salt_and_papper 실습자료에 있는 코드를 참고해서 구현하였다. src의 모든 pixel 에 대해 수행해야하기 때문에 2중 for 문을 돌며, mask 가 src 의 범위를 넘어서는 경우가 있기 때문에 정렬하고자 하는 size 를 재설정해주었다. Mask 크기가 아닌, row - (mask // 2) 의 값을 row_start의 좌표로, row + (mask // 2) 를 row_end 의 좌표로 하였고, col 역시 마찬가지로 (mask // 2) 의 값을 col 에서 빼준값과 더해준 값을 start 와 end 로 사용하였다. 그리고 이 때 각각의 값은 np.clip 함수를 사용해 0보다 작은 값은 0으로, src 의 최댓값인 h, w 를 넘어서면 h, w 값으로 설정해주었다. 이렇게 구한 좌표로 2차원의 mask 배열을 생성해주고, 정렬을 해주기 위해 np.flatten() 함수를 사용해 1차원배열로 만들어주었다. 이렇게 만든 1차원배열을 정렬해주고, 그 정렬된 배열에서 중간값 즉, (배열의 길이 // 2) 위치에 있는 값을 우리가 구하고자 하는 dst[row, col] 에 넣어주었다. 위와 같은 과정을 src 각각의 좌표에 수행시킨 후 dst 배열을 반환해주면 median filtering 과정이 끝이 난다. 사실 주어진 코드상에서 이 함수의 반환값이 dst.astype(uint8) 로 반환이 되었는데, 내가 완성시키고자 하는 이 프로젝트에서는 반환값을 uint8 로 바꾸는 과정을 my_normalize 함수로 수행하기 때문에 dst.astype(uint8) 로 반환하는 것이 아닌 아직 float type 의 값인 dst 로 반환하도록 수정해주었다.
이미지	<pre> dst = np.zeros((h, w)) for row in range(h): for col in range(w): ##### # TODO # TODO median filtering 코드 작성 # TODO median filtering 구현 시 openCV, np.median 사용금지 # TODO median filtering 구현 시 sorting 관련 내장 함수 사용 허용 # TODO 이전 실습 코드 참고 # median filtering 은 mask 의 값들을 정렬해서 중간값 사용 ##### r_start = np.clip(row - (msize // 2), 0, h) r_end = np.clip(row + (msize // 2), 0, h) c_start = np.clip(col - msize // 2, 0, w) c_end = np.clip(col + msize // 2, 0, w) mask = src[r_start:r_end, c_start:c_end] mask_1D = np.array(mask).flatten() # 1차원 배열을 정렬 sorted_array = np.sort(mask_1D) # 중앙값 계산 median = sorted_array[len(sorted_array) // 2] dst[row, col] = median return dst </pre>

함수이름	main 에서 RGB filtering
코드설명	이 부분은 main 함수에서 RGB 각각을 independent processing 을 수행하는 부분이다. 위에서 구해놓은 noise 가 낀 image 를 받아오는데 이 이미지는 color 이미지이므로 3개의 채널로 구성되어있는 이미지이다. 그래서 각각 R, G, B 채널로 나눠 independent processing 으로 구현해주었다. R, G, B 채널로는 noise_image 를 indexing 해주어 분리해주었고, 각각의 채널을 하나의 src 로 봐서 3번의 filtering 을 수행해주면 된다. 먼저 gaussian filtering 과정이다. my_get_Gaussian2D_mask() 함수를 사용해서 gaussian mask 를 생성해 my_filtering 함수에 src 와 mask 를 넣어 filtering 을 진행해주었다. 그 후 다시 하나의 채널로 합쳐야 color image 가 되므로 np.dstack 함수를 사용해 하나의 채널로 합치고, my_normalize() 함수를 사용해 정규화를 진행해주었다. Median filtering 과정은 gaussian filtering 과정처럼 다른 mask 가 필요한 것이 아니다. 그렇기 때문에 위에서 구현한 my_median_filtering() 함수를 사용해서 filtering 을 진행해주었다. 이 함수에는 src 와 mask size 를 인자로 넘겨주면 filtering 이 진행된 2차원 배열이 반환된다. 이 과정을 R, G, B 채널 각각에 대해 수행한 후 이 역시 1개의 채널로 다시 합쳐야 color image 가 된다, 그래서 np.dstack() 함수를 사용해 1개의 채널로 합쳐준 뒤 my_normalize() 함수를 사용해 정규화를 진행해주었다.
이미지	<pre>##### # TODO # TODO RGB에서 Gaussian, Median filter 진행 # TODO Filtering된 결과 이미지는 uint8형 타입이어야함 # -> (my_normalize 사용) ##### gaus2D = my_get_Gaussian2D_mask(msize=3, sigma=5) B = src_noise[:, :, 0] G = src_noise[:, :, 1] R = src_noise[:, :, 2] B_ = my_filtering(B, gaus2D) G_ = my_filtering(G, gaus2D) R_ = my_filtering(R, gaus2D) rgb_gaussian_dst = my_normalize(np.dstack((B_, G_, R_))) B_ = my_median_filtering(B, 3) G_ = my_median_filtering(G, 3) R_ = my_median_filtering(R, 3) rgb_median_dst = my_normalize(np.dstack((B_, G_, R_)))</pre>

함수이름	main 에서 YUV filtering
코드설명	이 부분은 위의 RGB filtering 과 동일하게 filtering 을 진행하지만, RGB image 를 YUV 로 변환 후 Y 만 filtering 을 진행하는 intensity only 방식으로 진행하는 부분이다. YUV 에서 Y 가 intensity 부분인데 이 부분만 filtering 을 진행한다. 이 부분에서도 gaussian filtering 과 median filtering 을 진행하므로 위에서 구현한 my_median_filtering 함수와 my_get_Gaussian2D_mask 로 구한 gaussian mask 를 사용한다. YUV 에서 indexing 으로 Y 채널을 분리하고, 이 채널만을 filtering 진행한다. Gaussian filtering 같은 경우는 my_filtering(Y, gaussian_mask) 로 filtering 을 진행해 구한 Y'과 기존 U, V 를 합쳐 1개의 채널이미지로 변환한다. Median filtering 역시 my_median_filtering(Y, mask_size) 로 filtering 을 진행한 후 구해진 Y' 과 기존 U, V 를 합쳐 하나의 채널로 구성된 image 로 만들어 구한다. YUV 에서 각각의 filtering 과정을 거쳐 구한 image 들은 my_normalize() 함수를 사용해 정규화를 진행해 마무리지었다. YUV 같은 경우 한개의 채널 이미지로 만든 것을 BGR 로 바꿔야 한다. 이 부분은 출력쪽에서 바꿔 출력하기 때문에 아래 코드에서 신경쓸 필요가 없다.
이미지	<pre>##### # TODO # TODO YUV에서 Gaussian, Median filter 진행 # TODO Filtering된 결과 이미지는 uint8형 타입이어야함 # -> (my_normalize 사용) # TODO yuv noise 이미지는 내장 모듈 사용 (cv2.COLOR_BGR2YUV) # TODO yuv_noise에서 첫번째 채널이 Y ##### gaus2D = my_get_Gaussian2D_mask(msize=3, sigma=5) yuv_noise = cv2.cvtColor(noise_image, cv2.COLOR_BGR2YUV) yuv_noise = yuv_noise / 255 Y = yuv_noise[:, :, 0] U = yuv_noise[:, :, 1] V = yuv_noise[:, :, 2] Y_ = my_filtering(Y, gaus2D) yuv_gaussian_dst = my_normalize(np.dstack((Y_, U, V))) Y = yuv_noise[:, :, 0] U = yuv_noise[:, :, 1] V = yuv_noise[:, :, 2] Y_ = my_median_filtering(Y, 3) yuv_median_dst = my_normalize(np.dstack((Y_, U, V)))</pre>

2. 결과 이미지

original	 A screenshot of a window titled "original_202102699" with standard macOS window controls (red, yellow, green buttons). The image inside shows two people in a red canoe on a calm river, surrounded by dense green foliage. The water reflects the boat and the surrounding trees.
Noise image	 A screenshot of a window titled "noise image_202102699" with standard macOS window controls. The image inside is the same scene as the original, but it is heavily corrupted with digital noise, appearing as a grainy, pixelated version of the original image where the details are obscured by random color patterns.

RGB Gaussian	
RGB Median	
YUV Gaussian	

YUV Median



3. 결과 분석

1. mask의 차이

본 과제에서는 두가지의 filtering 방법을 사용해 filtering 을 진행했다.

Gaussian filtering 과 median filtering 을 진행하였는데 gaussian mask 로 filtering 을 진행하면 주변 값을 반영해 filtering 을 진행해 smoothing 효과가 생기고, median mask 로 filtering 을 진행하면 주변값을 반영하는 것이 아니라서 smoothing 효과가 적용되지 않는다.

따라서 gaussian filtering 을 진행한 이미지가 median filtering 을 진행한 이미지보다 부드럽다.

2. Filtering 방법의 차이 (RGB or YUV)

RGB 각각의 채널에 filtering 을 진행하는 경우 색상채널에 filtering 을 진행하므로 색상의 보존이 이뤄지지 않을 수 있다.

하지만 YUV 방법처럼 filtering 을 진행하는 경우 이미지의 밝기 즉, intensity 만 filtering 을 진행하므로 색상은 보존하며 filtering 이 진행된다.

따라서 RGB 각각을 filtering 을 진행한 이미지보다 YUV에서 intensity 만 filtering 을 진행한 이미지가 색상 보존이 더 잘 된다.